Mutation-Guided Unit Test Generation with a Large Language Model

Guancheng Wang, *Member, IEEE,* Qinghua Xu, *Member, IEEE,* Lionel Briand, *Fellow, IEEE,* Kui Liu, *Member, IEEE,*

arXiv:2506.02954v4 [cs.SE] 22 Aug 2025

***Abstract*—Unit tests play a vital role in uncovering potential faults in software. While tools like EvoSuite focus on maximizing code coverage, recent advances in large language models (LLMs) have shifted attention toward LLM-based test generation. However, code coverage metrics—such as line and branch coverage—remain overly emphasized in reported research, despite being weak indicators of a test suite's fault-detection capability. In contrast, *mutation score* offers a more reliable and stringent measure, as demonstrated in our findings where some test suites achieve 100% coverage but only 4% mutation score. Although a few studies consider mutation score, the effectiveness of LLMs in killing mutants remains underexplored.**

In this paper, we propose MutGen, a mutation-guided, LLM-based test generation approach that incorporates mutation feedback directly into the prompt. Evaluated on 204 subjects from two benchmarks, MutGen significantly outperforms both EvoSuite and vanilla prompt-based strategies in terms of mutation score. Furthermore, MutGen introduces an iterative generation mechanism that pushes the limits of LLMs in killing additional mutants. Our study also provide insights into the limitations of LLM-based generation, analyzing the reasons for live and uncovered mutants, and the impact of different mutation operators on generation effectiveness.

***Index Terms*—Unit Test Generation, Mutation Testing, Large Language Models**

I. INTRODUCTION

UNIT test generation is a fundamental task in software engineering, crucial for ensuring code quality and reducing manual testing effort [10], [22], [30], [48]. With the rapid advancement of Large Language Models (LLMs), research has shifted from traditional techniques—such as random testing [12], [36], [50], [53], [54] and search-based methods [29], [58], [59], [65]—to LLM-based approaches for generating unit tests [27], [37], [66].

However, most existing work evaluates generated test cases primarily based on code coverage metrics, such as line or branch coverage. According to prior studies [11], [18], [39], high code coverage does not necessarily imply strong fault detection capability. Other research [9], [18], [28] suggests that the mutation score is a more reliable and meaningful metric for evaluating the effectiveness of test cases. Mutation testing introduces artificial faults, known as *mutants*, into the program by applying syntactic changes using mutation operators. The generated test cases are executed against the mutants, and if a test causes a mutant to fail (i.e., "kills" it), it is considered effective. The mutation score is the ratio of killed mutants to the total number of generated mutants.

Recent efforts have begun exploring mutation-based evaluation in the context of LLM-generated test cases. For example, Foster et al. [28] investigate regression testing scenarios in privacy-critical Kotlin applications such as WhatsApp and Instagram. They focus on generating test cases using LLMs based on existing ones, and on generating mutants using LLMs, which introduces challenges such as detecting equivalent mutants that no test can kill. Another work by Dakhel et al. [23] targets Python benchmarks and proposes a prompt-based approach to improve mutation score. However, they iteratively add a single mutant until they cannot kill the new mutant. Therefore, they do not attempt to converge towards a maximum mutation score to maximize fault detection. Moreover, their study does not analyze the reasons why live mutants remain undetected, nor does it consider uncovered mutants, which are challenges that we address in our study.

While these efforts represent promising first steps, the use of mutation testing as a core evaluation and improvement strategy for LLM-generated test cases remains underexplored. To address this gap, we propose MutGen, a mutation-guided LLM-based approach that explicitly aims to maximize mutation score. Specifically, a key feature of MutGen is that we incorporate mutation feedback (i.e., live and uncovered mutant information) into prompt construction to guide the LLM in generating more effective test cases. Further, as discussed in prior studies [55], [63], [68], incorporating a fixing step is crucial for improving test generation effectiveness. While prior work primarily focuses on improving metrics such as code coverage or test passing rate, our work explicitly targets the fault detection capability of test cases, measured by the mutation score. To this end, our approach introduces a fixing step that specifically addresses assertion failures in the generated test cases, caused by incorrect usage of assertion functions, including erroneous invocations and the use of inappropriate parameters.

Moreover, we observe that comments in the target code can mislead LLMs into producing irrelevant or incorrect tests. To address this, we introduce a code summarization step during preprocessing that replaces misleading comments with concise summaries, which are then embedded into the prompt.

Guancheng Wang is with the Research Ireland Lero Centre for Software, University of Limerick, V94 T9PX Limerick, Ireland (e-mail: guancheng.wang@ul.ie).

Qinghua Xu is with the Research Ireland Lero Centre for Software, University of Limerick, V94 T9PX Limerick, Ireland (e-mail: qinghua.xu@ul.ie).

Lionel Briand is with the University of Ottawa, Ottawa, ON K1N 6N5, Canada, and also with the Research Ireland Lero Centre for Software, University of Limerick, V94 T9PX Limerick, Ireland (e-mail: lbriand@uottawa.ca, lionel.briand@lero.ie).

Kui Liu is with Software Engineering Application Technology Lab Huawei, Hangzhou, China (e-mail: brucekuiliu@gmail.com).

Last, since achieving maximum mutation score is inherently difficult, we further introduce an iterative test generation strategy to push LLMs toward killing more live mutants. Once the mutation score converges, we analyze both uncovered and live mutants, as well as specific mutation operators that remain particularly challenging for the LLM. Like many recent studies [34], [60], [72], our approach focuses on function-level testing. While this prior work also generates test cases for functions from complex datasets such as Defects4J [41], they handle cross-module dependencies explicitly by just providing them in the prompts. In contrast, our work does not aim to address dependency resolution at this stage, but instead focuses on making significant improvements in mutation scores for function-level testing. Nevertheless, as discussed below, one of our datasets includes solutions to challenging algorithmic problems with complex program structures, providing a more complex dataset for evaluating the effectiveness of MutGen in generating test cases.

To evaluate MutGen, we use two datasets: HumanEval-Java, widely adopted in prior work [23], [61], [63], and a new LeetCode-Java corpus containing 100 randomly selected algorithmic problems with ground-truth solutions. The experimental results show that test cases generated by MutGen outperform both EvoSuite, a state-of-the-art search-based technique, and a vanilla LLM-based method in terms of mutation score. MutGen achieves mutation scores of 89.5% and 89.1% on HumanEval-Java and LeetCode-Java, respectively, across a total of 1,144 and 1,900 mutants.

To understand the contribution of each component in MutGen, we conduct an ablation study using three variants: MutGen-S (without summarization), MutGen-F (without the fixing step), and MutGen-MF (without mutation feedback). The results confirm the importance of each component in improving the overall mutation score.

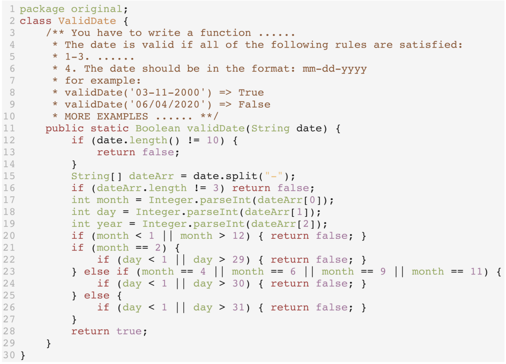
In summary, this paper makes the following contributions:

- We propose a mutation-guided, LLM-based approach (MutGen) for automatically generating test cases with high fault detection capability (i.e., mutation score), including an iterative generation process guided by mutation feedback to push the limits of LLMs in killing mutants.
- We evaluate our approach on a widely used benchmark dataset and a new dataset we created to increase diversity.
- We analyze the effectiveness of MutGen across different mutation operators and investigate the reasons behind the presence of live and uncovered mutants when MutGen reaches its maximum mutation score.

II. MOTIVATING EXAMPLE AND INFORMAL DESCRIPTION

This section goes through a simple example to informally describe our approach, MutGen, which we will then precisely describe and formalize in the next section. The example Java code under test shown in Figure 1, which validates a date format, includes descriptive comments and input-output examples. For clarity and to save space, we retain only the most relevant parts of the comments in the figure. Most LLM-based techniques prompt LLMs using the following standard prompt:

> *"Generate a JUnit test for the following method to achieve high branch and line coverage."*



```
 1 package original;
 2 class ValidDate {
 3     /** You have to write a function ......
 4      * The date is valid if all of the following rules are satisfied:
 5      * 1-3. ......
 6      * 4. The date should be in the format: mm-dd-yyyy
 7      * for example:
 8      * validDate('03-11-2000') => True
 9      * validDate('06/04/2020') => False
10      * MORE EXAMPLES ...... **/
11     public static Boolean validDate(String date) {
12         if (date.length() != 10) {
13             return false;
14         }
15         String[] dateArr = date.split("-");
16         if (dateArr.length != 3) return false;
17         int month = Integer.parseInt(dateArr[0]);
18         int day = Integer.parseInt(dateArr[1]);
19         int year = Integer.parseInt(dateArr[2]);
20         if (month < 1 || month > 12) { return false; }
21         if (month == 2) {
22             if (day < 1 || day > 29) { return false; }
23         } else if (month == 4 || month == 6 || month == 9 || month == 11) {
24             if (day < 1 || day > 30) { return false; }
25         } else {
26             if (day < 1 || day > 31) { return false; }
27         }
28         return true;
29     }
30 }
```

Fig. 1. Example code under test

When combined with the example code (including comments), LLMs can generate test cases that achieve high line and branch coverage. However, as demonstrated in prior work [9], [18], [28], high coverage does not necessarily imply strong fault detection capability, for example, when measured as mutation score. For instance, in our experiments, LLMs generate tests for the subject `id_81` from HumanEval-Java with 100% line and branch coverage, yet the corresponding mutation score is only 4%.

Listing 2 shows a representative test function generated by Llama-3.3. In practice, multiple such test functions may be generated, often with a variety of data inputs aimed at maximizing code coverage. However, we observe that most of these inputs fail to cover corner cases such as February 29th on leap years, likely due to the absence of such examples in the provided comment block. The comments in the target code may also introduce additional issues. For example, the comment begins with "You have to write a function…", which, when included as part of the prompt, may mislead the LLM into replicating the target code logic rather than generating a meaningful test function. Additionally, Llama-3.3 occasionally produces dates in incorrect formats such as `dd-mm-yyyy`, likely due to ambiguity or insufficient specification of the expected format in the comments provided within the program under test.

Further, when relying on mutation instead of code coverage, the generated inputs often fail to kill mutants effectively. For example, when the mutation operator changes a conditional boundary and alters line 24 to:

Listing 1. Example mutant introduced at line 24

```
if (day <= 1 || day > 30) return false;
```

This change introduces a fault: the original code accepts day == 1 as valid, while the mutant incorrectly rejects it. We observe that LLM-generated test cases, when prompted with the original code, tend to focus on representative invalid inputs (e.g., day = 0) and valid values far from the boundary (such as day = 2 or higher). For instance, the model commonly produces a test such as:

```
assertFalse(validDate("04-00-2025"));
```

which fails in both the original function and mutant. However, it frequently fails to generate:

```
assertTrue(validDate("04-01-2025"));
```

which is essential to kill the boundary-mutated version of the code.

Listing 2. Example test function generated by Llama-3.3

```java
@Test
public void testDayBoundaryInApril() {
  String date = "04-01-2025";
  assertTrue(ValidDate.validDate(date));
}
```

In summary, test cases generated using the standard prompt exhibit the following limitations:

1) Comments in the target code can mislead LLMs, resulting in the generation of ineffective or irrelevant test cases.
2) Given the presence of code mutants, LLMs struggle to understand how to generate tests that kill them effectively.

To address these issues, we propose MutGen, an approach that improves the fault detection capability of LLM-generated test cases via a two-stage process: preprocessing and generation, which are detailed in Section III-B and Sections III-C-III-E, respectively.

A. Preprocessing Stage

To address the first limitation above, MutGen removes comments from the target code and uses a prompt (shown in Figure 2) to instruct the LLM to summarize the method's purpose and input format. The prompt used is as follows:

Prompt Used for Summarizing

Summarize the following Java source code in 1-3 sentences, including its purpose and input format (if applicable).

`{{ source code }}`

Fig. 2. Prompt used for summarizing

In this example, Llama-3.3 returns the following summary:

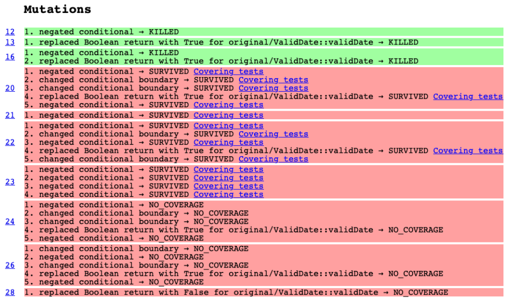
> *"This Java function, 'validDate', validates a given date string in the format "mm-dd-yyyy" and returns 'true' if the date is valid according to specific rules regarding month and day ranges, and 'false' otherwise. The input date string must be non-empty and follow the exact specified format. If any of these conditions are not met, the function will return 'false'."*

To address the second issue, we leverage mutation feedback to guide the LLM in generating test cases with higher fault detection capability. Before the generation stage, MutGen extracts mutation feedback from mutation reports produced by mutation testing tools. In this example, we use PITest [6] to generate a mutation report, as shown in Figure 3. From the report, MutGen retrieves key information, including the mutation location (line number), the mutant status (killed, survived, or uncovered), and the mutation operator applied to create the mutant. MutGen then integrates this feedback with the source code, forming a prompt (shown in Figure 4) that guides the LLM in generating more effective test cases during the generation stage. For example, the mutation feedback extracted from the report for the mutant shown in Listing 1 is as follows:

21. conditional boundary at line 24, uncovered:

```
if (day<= 1||day>30) return false;
```

This means that the existing tests fail to cover the 21st mutant, which is generated by applying a conditional boundary change at line 24, as shown on the second line.



```
Mutations

12  1. negated conditional → KILLED
13  1. replaced Boolean return with True for original/ValidDate::validDate → KILLED
16  1. negated conditional → KILLED
    2. replaced Boolean return with True for original/ValidDate::validDate → KILLED
20  1. negated conditional → SURVIVED Covering tests
    2. changed conditional boundary → SURVIVED Covering tests
    3. changed conditional boundary → SURVIVED Covering tests
    4. replaced Boolean return with True for original/ValidDate::validDate → SURVIVED Covering tests
    5. negated conditional → SURVIVED Covering tests
21  1. negated conditional → SURVIVED Covering tests
22  1. negated conditional → SURVIVED Covering tests
    2. changed conditional boundary → SURVIVED Covering tests
    3. negated conditional → SURVIVED Covering tests
    4. replaced Boolean return with True for original/ValidDate::validDate → SURVIVED Covering tests
    5. changed conditional boundary → SURVIVED Covering tests
23  1. negated conditional → SURVIVED Covering tests
    2. negated conditional → SURVIVED Covering tests
    3. negated conditional → SURVIVED Covering tests
    4. negated conditional → SURVIVED Covering tests
24  1. negated conditional → NO_COVERAGE
    2. changed conditional boundary → NO_COVERAGE
    3. changed conditional boundary → NO_COVERAGE
    4. replaced Boolean return with True for original/ValidDate::validDate → NO_COVERAGE
    5. negated conditional → NO_COVERAGE
26  1. changed conditional boundary → NO_COVERAGE
    2. negated conditional → NO_COVERAGE
    3. changed conditional boundary → NO_COVERAGE
    4. replaced Boolean return with True for original/ValidDate::validDate → NO_COVERAGE
    5. negated conditional → NO_COVERAGE
28  1. replaced Boolean return with False for original/ValidDate::validDate → NO_COVERAGE
```

Fig. 3. Example mutation report

B. Generation Stage

To achieve a high mutation score, MutGen employs a prompt augmented with mutation feedback, collected during the preprocessing stage, to guide Llama-3.3 in generating test cases, as illustrated below. For this HumanEval-Java subject, as shown in Figure 1, a single invocation of MutGen produces a test suite with 13 test cases, achieving a 70% mutation score across 30 mutants.

However, two of the generated test cases fail to execute due to runtime errors. To address this, MutGen incorporates a fixing mechanism that automatically repairs failing test cases, thereby improving both execution success rate and mutation score. In this example, MutGen successfully fixes one failing test case by invoking LLM with the following prompt, resulting in a 13% increase in mutation score.

To further enhance effectiveness, MutGen introduces an iterative generation strategy. In each iteration, additional test cases are generated to target the remaining live and uncovered mutants that were not killed by the previous test suite. After two iterations, for the running example, MutGen achieves a 100% mutation score. During this process, LLMs may occasionally generate test cases with method names that conflict with existing method names. For example, as shown in Listing 2, the LLM generates a test case for the boundary value 04-01-2025 in April. Later, it may generate another test case for 04-31-2025, also intended to test boundary conditions. However, both may be assigned the same method name, leading to naming conflicts. To resolve this, MutGen incorporates an additional guideline, specifically addressing method name conflicts, into the prompt used for fixing (shown in Figure 5).

In contrast, as demonstrated in Section IV, for this subject in HumanEval-Java, using the standard prompt introduced at

Prompt Used for Generation

```
# Unit Test Generator for Java using Junit4
## Source Code Summary {{summary}}
## Source File
    {{source code without comments}}
## Mutation Feedback
Analyze the live mutant to identify the defects; the generated test cases must look very different from the existing ones.
...
21. conditional boundary at line 24, uncovered:
if (day<=1||day>30) return false;
...
## Output Format in YAML
new_tests:
   - test_behavior: ...
       test_name: ...
       test_code: ...
       new_imports: ...
```

Fig. 4. Prompt used for generation

the beginning of this section, the LLM can only produce a test suite achieving a 53% mutation score, which remains unchanged even after four iterations.

Prompt Used for Fixing

```
## Failed Test
The following test failed in the test suite:
 {{ failed_test with error message }}
You are processing execution failures. Please match only one of these guidelines and then try to correct the failed tests according to the given error messages.
1. Direct Invocation:
• Always use assertion functions directly (e.g., assertEquals(value1, value2)) instead of prefixed calls like Assert.assertEquals(value1, value2).
2. Handling assertEquals Mismatches:
• If an error follows the pattern:
• test_file_name:line_number: expected:error_value but was:correct_value
• And involves assertEquals, replace the expected value with the correct one.
• Example: assertEquals(error_value, xxx); → assertEquals(correct_value, xxx);
3. Fixing assertTrue / assertFalse Errors:
• If an error follows the pattern:
• classname:line_number:null
• And involves assertTrue or assertFalse, swap the function name while keeping all parameters unchanged.
• Example: assertTrue(xxx); → assertFalse(xxx);
4. Handling ambiguous
• If an error has the pattern:
• reference to assertEquals is ambiguous
• And involves assertEquals, add type casts to both parameters based on their respective types.
• Example: assertEquals(expectedValue, actualValue); → assertEquals((long) expectedValue, (long) actualValue);
5. Method name conflicts
• If an error follows the pattern: method is already defined in the class,
• only append a number to the method name.
• Example: public void test_null() → public void test_null1();
6. Other errors
• Try to fix the errors according to the error messages.
## Output Format in YAML
The same applied to Figure 4
```

Fig. 5. Prompt used for fixing

III. OUR APPROACH

In this section, we precisely describe and formalize our proposed approach, MUTGEN. As shown in Figure 6, MUTGEN takes a source program as input and returns test cases aimed at improving its fault-detection effectiveness by maximizing the test suite's mutation score. We highlight all data flows in red in the figure. Steps 1, 2, and 5 are described in Sections III-B, III-C, and III-D, respectively. We do not include Section III-E as it reuses the components already shown in the figure. We begin with a high-level overview of MUTGEN, followed by detailed and formal descriptions of each individual component.

A. Overview

Let $\mathcal{M} = \{m_1, m_2, \ldots, m_n\}$ denote the set of mutants generated from the program under test. The mutation score $\mu(T)$ of a test suite T is defined as:

$$\mu(\mathcal{T}) = \frac{|\{m \in \mathcal{M} \mid \mathcal{T} \text{ kills } m\}|}{|\mathcal{M}|}$$

We aim to generate a test suite $\mathcal{T}$ such that $\mu(\mathcal{T})$ is maximized. To achieve this, we propose a two-stage process based on prompting an LLM, namely preprocessing and generation.

The preprocessing stage prepares information to guide test case generation, including code summarization and mutation feedback, as detailed in Section III-B. In the generation stage, the LLM is prompted with mutation feedback, such as descriptions of mutation operators and diff-like representations of mutants, to generate test cases.

Let $\mathcal{R} = \{r_1, r_2, \ldots, r_k\}$ denote the set of mutation feedback entries (i.e., killed, live, uncovered), where each r_i is a tuple $\langle \text{id}_i, m_i, s_i, \text{op}_i \rangle$, respectively representing the mutant identifier, the mutated statement, the mutant status, and the applied mutation operator. MUTGEN first generates an initial test suite $\mathcal{T}$:

$$\mathcal{T} = \text{LLM}(\text{Prompt}_{\text{mut}}(\mathcal{R}))$$

Here, as further described in the following sections, $\text{Prompt}_{\text{mut}}$ is a structured prompt that encodes details of the mutation operators and specific mutant instances, as illustrated in the example $\text{Prompt}_{\text{mut}}$ shown in Figure 4.

After executing $\mathcal{T}$, MUTGEN collects a set $\mathcal{F} = \{f_1, ..., f_q\}$ consisting of all test cases that encountered execution failures and corresponding error messages. For each f_i, MUTGEN constructs a second prompt $\text{Prompt}_{\text{fail}}$ (as illustrated in the example shown in Figure 5) and invokes the LLM to generate a set of fixed test cases $\mathcal{T}_{fix}$:

$$\mathcal{T}_{fix} = \{\text{LLM}(\text{Prompt}_{\text{fail}}(f_i)) \mid f_i \in \mathcal{F}\}$$

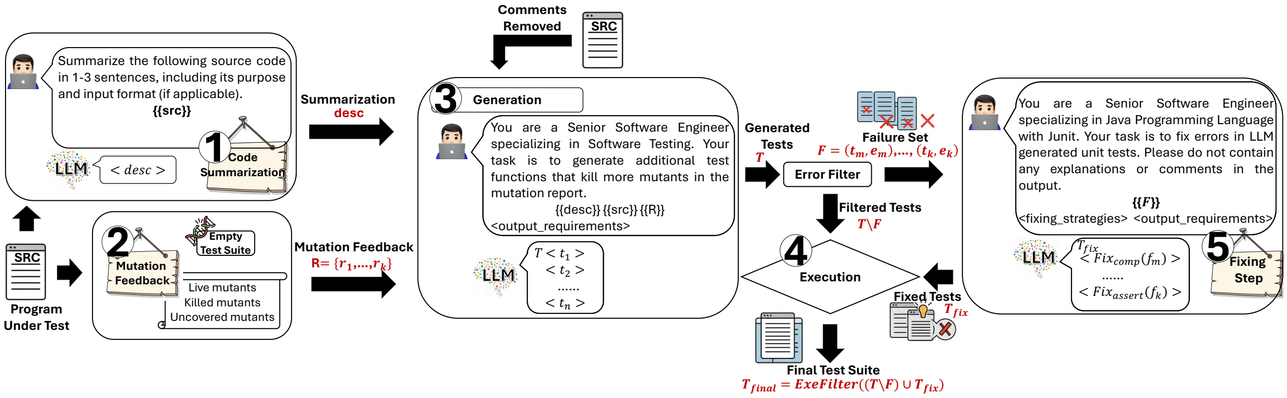
The final test suite consists of all passing test cases obtained by combining the filtered tests and the fixed tests. We denote this using the function `ExeFilter`, as follows:

$$\mathcal{T}_{\text{final}} = \texttt{ExeFilter}((\mathcal{T} \setminus \mathcal{F}) \cup \mathcal{T}_{\text{fix}})$$

This process will be detailed in Section III-D.

Our objective is to maximize the mutation score:

$$\mathcal{T}^* = \arg\max_{\mathcal{T}} \mu(\mathcal{T})$$



Summarize the following source code in 1-3 sentences, including its purpose and input format (if applicable).
{{src}}

You are a Senior Software Engineer specializing in Software Testing. Your task is to generate additional test functions that kill more mutants in the mutation report.
{{desc}} {{src}} {{R}}
<output_requirements>

You are a Senior Software Engineer specializing in Java Programming Language with Junit. Your task is to fix errors in LLM generated unit tests. Please do not contain any explanations or comments in the output.
{{F}}
<fixing_strategies> <output_requirements>

Fig. 6. MutGen Overview

Our assumption, as reported in the mutation testing literature [11], [18], is that by maximizing the mutation score, we also maximize fault detection effectiveness. In any case, achieving mutation coverage is known to be a more stringent criterion than line or branch coverage [28] and therefore enables the generation of more effective test suites. However, as revealed in recent work [32], finding an optimal prompt for LLMs remains a significant challenge, and we introduce an approximate and iterative solution, detailed in Section III-E, which we investigate empirically and whose limitations we analyze, for example, in terms of mutation operators that are difficult to kill.

B. Preprocessing

The preprocessing stage prepares information to guide the test case generation.

Code Summarization. As illustrated in Figure 6 and the example in Section II-A, the prompt instructs the LLM to summarize the purpose and the input format of the target code, resulting in a natural language description denoted as *desc*.

Mutation Feedback. Additionally, we apply a mutation testing tool to the code under test to generate mutation feedback. Since mutation testing tools rely on an existing test suite, we first create an empty test suite by mimicking a test function that returns directly. We then collect the mutation analysis results produced by the tool. As illustrated in Figure 6, the resulting mutation report includes information on live mutants, killed mutants, and uncovered mutants. Mutation feedback can be obtained by associating this information with the corresponding code.

As shown in the example in Section II-A, the corresponding mutation feedback entry is represented as:

$$\begin{aligned} r_{21} = \langle & 21, \\ & if\ (day \leq 1 \| day > 30)\ return false;, \\ & uncovered, \\ & change\ conditional\ boundary\ at\ line\ 24\ \rangle \end{aligned}$$

C. Prompting with Mutation Feedback

We define the prompt function with mutation feedback $\text{Prompt}_{\text{mut}}$ as:

$$\text{Prompt}_{\text{mut}} : \mathcal{R} \to \text{String}$$

$$\text{Prompt}_{\text{mut}}(\mathcal{R}) = \texttt{Format}\left(desc, \bigcup_{r_i \in \mathcal{R}} r_i\right)$$

The function $\texttt{Format}(\cdot)$ organizes the given parameters into a structured prompt suitable for the LLM.

As illustrated in Figure 6 and the example in Section II-B, the prompt constructed by MutGen includes a natural language code summarization (*desc*), the source code excluding comments (*src*), and the associated mutation feedback ($\mathcal{R}$). In addition, MutGen explicitly specifies the expected output format from the LLM, as detailed in the prompt example Figure 4 shown in Section II-B.

D. Fixing Execution Errors

After the initial generation stage, some test cases may fail due to errors in their code. MutGen further introduces a *fixing step* that applies a targeted prompt, as illustrated in the example prompt shown in Figure 5, to repair them.

We categorize these failures into two types:

- **Assertion Failures:** Test cases that can pass compilation but fail to execute due to violated assertions or incorrect outputs.
- **Compilation Errors:** Test cases that cannot be compiled due to syntax errors, missing types, unresolved references, or similar issues.

As shown in prior work [72], advanced LLMs are generally capable of producing syntactically correct test cases. However, they often suffer from runtime errors, among which assertion failures are widespread. In this paper, which targets unit testing scenarios, we focus on addressing assertion failures. To this end, MutGen incorporates four fix strategies into the prompt: Direct Invocation, Handling AssertEquals Mismatches, Fixing assertTrue/assertFalse Errors, and Handling Ambiguous Cases. For potential compilation errors, we employ a fifth strategy, which proactively avoids these issues during the process described in Section III-E. For other errors, MutGen adopts a sixth strategy, instructing the LLM to actively repair them rather than ignoring them. All fix strategies are listed in the prompt shown in Figure [5].

We partition the set of failures into compilation and assertion failures such that

$$\mathcal{F} = \mathcal{F}_{\text{comp}} \cup \mathcal{F}_{\text{assert}}, \quad \mathcal{F}_{\text{comp}} \cap \mathcal{F}_{\text{assert}} = \emptyset.$$

Each failing test case is associated with its corresponding error message, forming a tuple $f_i = (t_i, e_i)$, where t_i is the original test case and e_i is the compiler or runtime error message.

We define a prompt construction function:

$$\text{Prompt}_{\text{fail}}(f_i) = \texttt{Format}(t_i, e_i),$$

which creates a structured natural language prompt, as illustrated in Figure [5].

We define the specialized fix functions as:

$$\begin{aligned} \text{Fix}_{\text{comp}}(f_i) &= \text{LLM}(\text{Prompt}_{\text{fail}}(f_i)) & \text{for } f_i \in \mathcal{F}_{\text{comp}}, \\ \text{Fix}_{\text{assert}}(f_i) &= \text{LLM}(\text{Prompt}_{\text{fail}}(f_i)) & \text{for } f_i \in \mathcal{F}_{\text{assert}}. \end{aligned}$$

The outputs of these functions are the repaired test sets:

$$\mathcal{T}' = \{\text{Fix}_{\text{comp}}(f_i) \mid f_i \in \mathcal{F}_{\text{comp}}\},$$
$$\mathcal{T}'' = \{\text{Fix}_{\text{assert}}(f_i) \mid f_i \in \mathcal{F}_{\text{assert}}\}.$$

We denote the final set of fixed test cases as:

$$\mathcal{T}_{\text{fix}} = \mathcal{T}' \cup \mathcal{T}''.$$

E. Pushing the Limit: Iterative Generation

We propose an iterative generation process that leverages mutation feedback and failed test cases to progressively enhance the quality and effectiveness of the generated tests. At each iteration i, MutGen obtains a working test suite $\mathcal{T}_i$. Before proceeding to the next iteration, MutGen evaluates $\mathcal{T}_i$ to obtain mutation feedback $\mathcal{R}_i$. The LLM is re-invoked with $\text{Prompt}_{mut}(\mathcal{R}_i)$ to generate an initial set of new test cases. The failing cases are filtered as $\mathcal{F}_i$ and, for each $f_j \in \mathcal{F}_i$, the LLM is subsequently invoked with $\text{Prompt}_{fail}(f_j)$ to fix them. MutGen obtains $\Delta\mathcal{T}_i$ as the set of test cases that successfully execute. The updated test suite $\mathcal{T}_{i+1}$ is defined as:

$$\mathcal{T}_{i+1} = \mathcal{T}_i \cup \Delta\mathcal{T}_i,$$

The process continues until convergence (i.e., no significant improvement in mutation score) or until a predefined iteration limit is reached.

IV. EVALUATION

As previously discussed, MutGen is designed to enhance the fault-detection capability of LLMs in automated test generation by incorporating mutation feedback. To evaluate its effectiveness, we compare MutGen with three baselines: **EvoSuite**, the state-of-the-art search-based technique; **EvoSuite$_{\text{mut}}$**, a variant that replaces coverage-based fitness functions with strong mutation as described in [31]; and **Gen$_{\text{vanilla}}$**, a prompting-based approach that utilizes an LLM without incorporating mutation feedback and leveraging the comments provided in the program under test through summarization (as illustrated in Figure [4]). We adopt Llama-3.3 70B as the base model in our evaluation due to its recent release, strong effectiveness on code-related tasks [33], and open-source accessibility, which ensures reproducibility and ease of deployment in both research and practical scenarios.

In addition to effectiveness comparisons, we analyze the limitations of LLM-based test generation by examining killing ratios across different mutation operators. This allows us to identify which types of mutants are particularly challenging for LLMs to kill and offer insights into potential areas for improvement.

Finally, we conduct an ablation study to evaluate the contribution of each component within MutGen to the overall effectiveness.

To sum up, our evaluation addresses the following research questions.

- **RQ1:** How does MutGen perform in generating test cases with high fault detection capability (i.e., mutation score) compared to EvoSuite, EvoSuite$_{\text{mut}}$, and Gen$_{\text{vanilla}}$?
- **RQ2:** Which mutation operators remain challenging for LLM-based test generation and for what reasons?
- **RQ3:** What is the individual contribution of each component of MutGen to the overall effectiveness?

A. Experiment Setup

Subjects. In our study, we primarily focus on method-level subjects, where each subject is a single Java class containing only one target method. We generate unit test cases specifically for this method. We use a widely adopted benchmark dataset that has been used in prior publications [40], [61], [63], [75], and we also create an additional dataset to enable a more diverse evaluation. More specifically, we include the following subjects:

a) HumanEval-Java: Following prior work [40], [61], [63], [75], we adopt the HumanEval dataset [14], [19] as one of our evaluation benchmarks. Specifically, we utilize all 160 Java subjects from the dataset. For each subject, we apply EvoSuite with its default configuration to automatically generate a test suite. We then evaluate the mutation score of the generated test suite using PITest [6]. Subjects where MutGen and EvoSuite both achieved a 100% mutation score are discarded, resulting in 104 Java subjects for our evaluation. Indeed, these subjects do not help us distinguish the two approaches.

b) Leetcode-Java: We search GitHub using the keyword `"Leetcode Java"` and select the top two repositories based on the *Best match* sorting criterion. These repositories contain Java solutions to algorithmic problems from LeetCode [3], which are categorized into three difficulty levels: *Easy*, *Medium*, and *Hard*. We exclude all solutions corresponding to Easy problems. We collect all Java source files for the remaining *Medium* and *Hard* problems and organize them into a unified package structure following the convention used in HumanEval-Java. We use the Maven build system to compile the project and remove any subjects that failed to compile. For each valid subject, we apply EvoSuite with its default settings to generate a test suite. Subjects for which the generated test suite achieved a 100% mutation score by MutGen and EvoSuite are once again discarded. Finally, for computational feasibility, we randomly select 50 subjects from each of the *Medium* and *Hard* sets, resulting in a total of 100 Leetcode-Java subjects used in our evaluation.

Metrics. Since our work focuses on enhancing the effectiveness of LLM-generated test cases, we adopt the **mutation score**, calculated as the ratio of killed mutants to the total number of mutants, as the primary evaluation metric. A mutant is considered *killed* if at least one generated test case causes a behavioral difference between the original program and its mutated version, typically resulting in a failed assertion or exception. In addition to the mutation score, we also report **branch coverage** and **line coverage**, in line with previous work [8], [20].

Process. To address **RQ1**, we use all subjects from both datasets. We begin by generating test cases with both $\text{Gen}_{\text{vanilla}}$ and MutGen through a single end-to-end execution, recording the execution time and the number of tokens consumed. We then apply EvoSuite and $\text{EvoSuite}_{\text{mut}}$ to generate test cases for each subject, setting their time budget to the larger average processing times of MutGen on the two datasets. It is important to note that, unlike prior work [28], our approach generates tests entirely from scratch, without relying on any existing test cases. We evaluate each test suite using two structural coverage metrics, **branch coverage** and **line coverage**, measured by the JaCoCo tool [2]. Additionally, we use PITest [6] with its DEFAULTS mutation operator group to measure the **mutation score** of each test suite. While PITest also offers a STRONGER operator group, it differs from DEFAULTS by only two additional mutation operators, whose applicability is rather limited. For example, the *Remove Conditionals* operator only targets equality checks. As for the ALL group provided by PITest, the extra mutation operators in this group generally generate mutants that are relatively easy to kill, making them less informative for our evaluation. To assess the statistical significance of the improvements achieved by our approach in terms of effectiveness, we performed non-parametric hypothesis testing using Vargha and Delaney's A12 statistic [13], [64]. The A12 effect size was computed based on the mutation score, which ranges from 0 to 1: a value of 0.5 indicates no difference (i.e., results are due to chance), while a value greater than 0.5 suggests that MutGen is more likely to outperform the compared approach. Conversely, a value less than 0.5 indicates the opposite. We also conducted a paired-sample Wilcoxon signed-rank test on both coverage and mutation score to evaluate whether MutGen achieves statistically significant improvements on these metrics.

To address **RQ2**, we analyze the mutation operators involved in the evaluation. As a preliminary experiment, we first randomly select 10 subjects from each dataset and perform seven rounds of end-to-end test generation. In each round, MutGen generates additional test cases based on the previously generated ones. By tracking the mutation scores across iterations, we observe that the scores tend to converge after the fourth iteration. Therefore, we set the maximum number of repetitions to four in our experiments. We collect results at the fourth iteration, and analyze the distribution of mutation operators across all generated mutants and compute the *killing ratio* for each operator, that is, the proportion of killed mutants relative to the total number of mutants produced by that operator. This allows us to identify which mutation operators tend to produce mutants that are difficult or easy for LLM-generated test cases to kill. In our evaluation, mutation testing is conducted using PITest, which employs 11 mutation operators as described in [1]. For mutation operators with low killing ratios, we investigate why LLMs struggle to kill these mutants. Due to space constraints, we do not present a detailed analysis for each mutation operator or for all live and uncovered mutants. Instead, we analyze a subset of cases and identify several factors contributing to the inability of LLMs to kill certain mutants. Nevertheless, our findings offer valuable insights and highlight important directions for future research in this area.

To address **RQ3**, we conduct an ablation study to evaluate the contribution of each core component in MutGen. Specifically, MutGen consists of three main components: (1) extracting and incorporating code summarization into prompts, (2) applying the fixing step to repair invalid test cases, and (3) extracting and incorporating mutation feedback into prompts. In this study, we disable one component at a time, resulting in three variants MutGen-S (using the original code comments instead of summarization), MutGen-F (omitting the fixing step), and MutGen-MF (excluding mutation feedback). For each variant, we perform four iterations of test case generation and measure mutation score to assess the impact of each component on the effectiveness of MutGen.

Implementation. We describe important implementation details of MutGen and our evaluation setup. All subjects used in the evaluation are built using the Maven build system. We follow official guidelines to integrate EvoSuite, JaCoCo, and PITest into the Maven pipeline for automated test generation, code coverage measurement, and mutation testing, respectively. Our approach, MutGen, is implemented in Python and built on top of the Mutahunter framework [4]. Specifically, we reuse components from Mutahunter for prompt handling, LLM interfacing, and response processing. We use Ollama [5] to locally deploy the Llama-3.3 70B model [33] with the default configuration.

The evaluation is conducted on a Linux server equipped with a 56-core CPU, 125 GB of RAM, a single NVIDIA RTX 6000 Ada GPU, and running Ubuntu Linux 24.04.1 LTS. Our tool and benchmarks can be found at: https://anonymous.4open.science/r/MUTGEN-4A06.

B. Results and Analysis

a) Comparison with EvoSuite, EvoSuite_{mut}, and $\text{Gen}_{vanilla}$: Table I presents the overall results of EvoSuite, $\text{EvoSuite}_{\text{mut}}$, $\text{Gen}_{\text{vanilla}}$, and our approach MutGen on the two datasets: HumanEval-Java and Leetcode-Java. The best results for each metric and dataset are highlighted.

On HumanEval-Java, for both **line coverage** and **branch coverage**, MutGen outperforms EvoSuite, $\text{EvoSuite}_{\text{mut}}$ and $\text{Gen}_{\text{vanilla}}$. On Leetcode-Java, we have the opposite since

TABLE I
Summary of Results

Approach	Dataset	Line Cov	Branch Cov	Mutation Score	#Mutants per Subject	A12 effect size	Time (secs)	#Tokens
EvoSuite	HumanEval-Java	95.6%	93.4%	69.5%	11	0.759	150	1240
	Leetcode-Java	**99.0%**	**98.9%**	58.9%	19	0.899	150	956
EvoSuite$_{\text{mut}}$	HumanEval-Java	95.4%	93.4%	67.4%	11	0.744	150	1232
	Leetcode-Java	98.1%	98.7%	58.1%	19	0.889	150	940
Gen$_{\text{vanilla}}$	HumanEval-Java	96.2%	92.8%	77.9%	11	0.650	74.1	846
	Leetcode-Java	96.3%	92.7%	69.9%	19	0.734	76.3	1232
MutGen	HumanEval-Java	**98.3%**	**95.8%**	**89.5%**	11	–	125.9	899
	Leetcode-Java	98.4%	94.8%	**89.1%**	19	–	149.4	1629

EvoSuite achieves the highest scores for coverage, surpassing both LLM-based techniques. For both datasets, however, differences are relatively small, below three percentage points. Although MutGen is primarily designed to improve mutation score, the results also demonstrate that it consistently outperforms Gen$_{\text{vanilla}}$ in terms of line and branch coverage. This supports previous observations [28] that improvements in mutation score can also lead to enhanced code coverage.

Regarding the **mutation score**, PITest generates an average of 11 and 19 mutants per subject on HumanEval-Java and Leetcode-Java, respectively, resulting in a total of 1,144 and 1,900 mutants across the two datasets. MutGen achieves mutation scores of 89.5% and 89.1% on the two datasets. Compared to EvoSuite, this represents large improvements of 28.8% and 51.3%, respectively, corresponding to around 20 and 30 percentage points. Compared to Gen$_{\text{vanilla}}$, MutGen also significantly improves the mutation score, though to a lesser extent. Additionally, MutGen also surpasses EvoSuite$_{\text{mut}}$, which specifically aims to maximize the mutation score, to an even larger extent. Indeed, surprisingly, we observe that EvoSuite slightly outperforms EvoSuite$_{\text{mut}}$. This might be due to the latter being developed 10 years ago [30], without any further maintenance. For example, we found that the mutation operators used by EvoSuite$_{\text{mut}}$ are limited, resulting in poor performance of the generated test cases when evaluated by PITest. Another tentative explanation is that, to ensure fair comparisons, recall that we set the time budget as the maximum budget used by MutGen, but a large budget may be necessary for EvoSuite$_{\text{mut}}$.

Furthermore, we report the *A12 effect size* [13] on mutation scores in the last column of Table I. For the comparison between EvoSuite and MutGen, the A12 values are 0.759 and 0.899, indicating that MutGen has a 75.9% and 89.9% probability of achieving higher mutation scores than EvoSuite on HumanEval-Java and Leetcode-Java, respectively. Similarly, when compared with Gen$_{\text{vanilla}}$, the A12 values are 0.650 and 0.734, suggesting that MutGen outperforms Gen$_{\text{vanilla}}$ with 65.0% and 73.4% probabilities on the same datasets. Additionally, based on the results of the paired-sample Wilcoxon signed-rank test, we found that MutGen achieves statistically significant improvements ($\alpha = 0.05$) in mutation scores compared with EvoSuite, EvoSuite$_{\text{mut}}$, and Gen$_{\text{vanilla}}$. However, there is no statistically significant difference in coverage, as most subjects already achieve near 100% coverage across all approaches.

In terms of overhead, MutGen consumes fewer tokens on HumanEval-Java but more tokens on Leetcode-Java. The Leetcode-Java subjects are solutions to medium- and hard-level algorithmic problems, with more complex logic than those in HumanEval-Java. As a result, PITest generates more mutants for them (19 vs. 11), and Llama-3.3 requires more test cases to kill all mutants. Compared with Gen$_{\text{vanilla}}$, EvoSuite, and EvoSuite$_{\text{mut}}$, in Leetcode-Java, MutGen consumes more tokens while achieving higher mutation scores as well as better line and branch coverage. In contrast, on HumanEval-Java, MutGen consumes fewer tokens than EvoSuite and EvoSuite$_{\text{mut}}$ while still achieving superior performance in both coverage and mutation score. However, differences of that scale in the number of tokens do not have practical implications.

Overall, results suggest that MutGen helps significantly improve mutation scores over alternatives, with large differences, at the expense of slightly lower code coverage when compared to EvoSuite, for the most complex subject (Leetcode-Java). This may be expected as EvoSuite is designed to maximize coverage. For the other subject (HumanEval-Java), the reverse trend is observed. Even when EvoSuite is specifically configured as EvoSuite$_{\text{mut}}$ to maximize mutation scores [30], it performs slightly worse than EvoSuite on both datasets. Therefore, to optimize mutation scores and, thus, fault detection, we recommend the use of MutGen, though the impact on coverage is unclear and probably practically negligible.

TABLE II
Mutant Status Counts per Operator on HumanEval-Java
(Sorted by Total)

Operator	**K**	**L**	**NoCOV**	**Total**	**Killed Ratio**
NegateConditionals	398	7	3	408	97.5%
Math	209	11	3	223	93.7%
ConditionalsBoundary	186	18	4	208	89.4%
EmptyReturns	71	1	4	76	93.4%
TrueReturns	45	7	2	54	83.3%
Increments	50	2	1	53	94.3%
PrimitiveReturns	29	4	1	34	85.3%
FalseReturns	26	0	1	27	96.3%
VoidMethodCalls	18	7	1	26	69.2%
NullReturns	19	0	0	19	100.0%
InvertNegatives	16	0	0	16	100.0%
Summary	1067	57	20	1144	93.3%

b) Effectiveness of MutGen *on Different Mutation Operators:* To evaluate the effectiveness and limitations of MutGen in killing mutants, we analyze the killed ratio across different mutation operators (described in [1]) after running MutGen for four iterations. Tables II and III present the mu-

TABLE III
MUTANT STATUS COUNTS PER OPERATOR ON LEETCODE-JAVA (SORTED BY TOTAL)

Operator	K	L	NoCOV	Total	Killed Ratio
Math	766	8	14	788	97.2%
NegateConditionals	474	5	14	493	96.1%
ConditionalsBoundary	288	11	1	301	95.7%
PrimitiveReturns	97	14	2	113	85.8%
VoidMethodCalls	58	2	7	67	86.6%
Increments	46	7	0	53	86.8%
EmptyReturns	24	0	2	26	92.3%
TrueReturns	15	0	11	26	57.7%
NullReturns	17	0	2	19	89.5%
FalseReturns	13	0	0	13	100.0%
InvertNegatives	1	0	1	2	50.0%
Summary	1799	47	54	1900	94.7%

tant status counts for each mutation operator on HumanEval-Java and Leetcode-Java, respectively. In both tables, Column **K** indicates the number of killed mutants for each corresponding mutation operator. Column **L** shows the number of live mutants (covered by not killed), while Column **NoCOV** represents the number of mutants that are not covered by any test cases. Column **Killed Ratio** denotes the proportion of killed mutants relative to the total number of mutants.

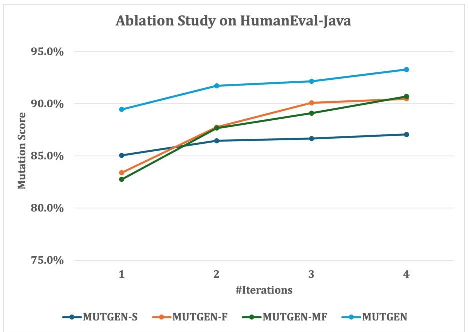
We observe that, for frequently applied mutation operators, such as `Math` and `NegateConditionals`, MUTGEN is able to kill the corresponding mutants across iterations progressively. After four iterations, MUTGEN achieves a high killed ratio—for instance, 97.2% for `Math` on Leetcode-Java. The remaining 2.8% of mutants are mostly uncovered, meaning they are not exercised by any generated test cases.

In contrast, mutation operators that appear less frequently tend to exhibit lower effectiveness. For example, `VoidMethodCalls` shows a killed ratio of 69.2% on HumanEval-Java, and `TrueReturns` achieves only 57.7% on Leetcode-Java, representing the weakest effectiveness among all mutation operators in terms of killed ratio.

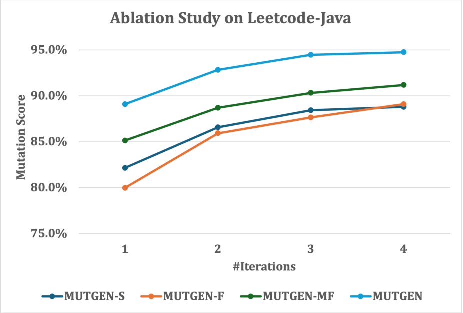
For the uncovered mutants, the primary reasons are the presence of dead code or the inability of the generated test cases to reach the mutated statements, even after incorporating mutation feedback. This suggests that further improvements in mutation score may require more fine-grained information and targeted guidance. Exploring how to provide such detailed feedback to LLMs effectively remains an open direction for future research.

For live mutants, most mutation operators alter the original program logic. For example, the `MATH` operator may mutate a statement `a = b + c;` to `a = b - c;`, changing the value of the variable `a`. If `a` is not critical to the program's behavior, the LLM may fail to recognize the impact of the mutation and thus generate test cases that do not expose the fault. This observation further highlights the need for more fine-grained semantic information to guide LLMs in identifying and targeting meaningful behavioral changes, thereby improving their ability to detect live mutants.

c) Ablation Study: Figure 7 presents the mutation scores achieved by the ablation variants—MUTGEN-S, MUTGEN-F, MUTGEN-MF—and our approach MUTGEN across four iterations. Specifically, Figure 7(a) shows the results on HumanEval-Java, while Figure 7(b) reports the results on Leetcode-Java. The horizontal axis represents the number of iterations, and the vertical axis denotes the mutation score.



(a) Results on HumanEval-Java



(b) Results on Leetcode-Java

Fig. 7. Mutation Score Changes over 4 Iterations: Ablation Study on Both Datasets.

Based on the mutation scores obtained at the fourth iteration, code summarization appears to have the most significant impact on the effectiveness of MUTGEN. As discussed in Section II, code comments can sometimes mislead LLMs or omit essential details—such as input formats—that are crucial for guiding test case generation. The fixing stage also shows a notable impact on effectiveness, particularly on the LeetCode-Java dataset, where many generated test cases initially fail to execute successfully. Prior work has observed that LLM-generated test cases often suffer from low execution success rates. After applying fixes, however, a substantial portion of these test cases become valid and contribute to the test objectives. Our evaluation confirms that the fixing stage contributes meaningfully to improved mutation scores.

Lastly, mutation feedback also plays a role in enhancing effectiveness. As revealed in RQ2, while many mutants generated by common mutation operators are relatively easy for LLMs to kill, mutation feedback still provides effectiveness gains by guiding the model more effectively. Future work may further investigate how mutation feedback can be tailored to help LLMs kill harder-to-detect mutants, and which mutation operators benefit the most from such guidance. We leave this exploration as a direction for future research.

C. Threats to Validity

The threat to internal validity mainly lies in the correctness of the implementation of MUTGEN and the experimental scripts. To reduce this threat, we have carefully reviewed our code.

The threat to external validity mainly lies in the subjects and the target approaches. For subject selection, we reused benchmarks from existing publications and further enhanced subject diversity by evaluating our approach on 100 additional subjects. These were randomly sampled from a corpus of LeetCode algorithm solutions. In the future, we will evaluate MUTGEN on more complicated subjects, e.g., Defects4J. Notably, Defects4J contains real-world defects, which can provide a stronger assessment of the practical effectiveness of our approach, compared to the synthetic defects simulated via mutation testing in this study.

Regarding the baseline approaches, we selected two representative methods: (1) EVOSUITE, a widely-used state-of-the-art search-based testing tool, and (2) $\text{GEN}_{\text{vanilla}}$, a vanilla LLM-based prompt approach. These baselines allow us to evaluate whether incorporating mutation feedback in MUTGEN significantly improves fault detection capability, as measured by mutation score. In addition to the above compared methods, we also compared MUTGEN with a variant of EVOSUITE. Following published guidelines [31], we enabled the fitness function for strong mutation testing in EVOSUITE. We conducted experiments on the two datasets using only the strong mutation fitness function. The average mutation scores for both datasets were 67.4% and 58.1%, respectively, which are similar to the results reported in our paper using the EVOSUITE default setting. Although we use Llama-3.3 as the base LLM in this study, our approach is not tied to any specific model. MUTGEN is model-agnostic by design, and evaluating its performance with other LLMs is left as future work.

Several factors may affect the construct validity of our evaluation.

Randomness. Randomness may impact the effectiveness of EVOSUITE, MUTGEN, and its variant. To mitigate this, and considering computational cost, we ran each tool on every subject three times and reported average results. We also performed statistical significance tests and calculated effect sizes, as detailed in Section IV-B.

Equivalent Mutants. Our approach uses the `PITest` mutation testing framework, which includes mechanisms to reduce the creation of equivalent mutants. While this mitigates the problem to some extent, the remaining equivalent mutants can reduce the efficiency of MUTGEN and prevent it from achieving a 100% mutation score.

Oracle Problem. We assume the original code is correct and only address assertion errors that cause Maven build failures. This simplification can lead to the generation of incorrect test oracles and result in false positives.

Data Leakage. While some public code in benchmark datasets may appear in the LLM's training data (e.g., Llama-3.3), the specific mutants we generate are unlikely to be memorized. Moreover, we compare MUTGEN with a vanilla prompting baseline, and the observed performance gains suggest that our improvements stem from prompt engineering and strategy design rather than memorization.

V. RELATED WORK

Mutation-Guided Test Generation. Recent studies have questioned the sufficiency of coverage as an evaluation criterion. As shown in [11], [18], [39], and pointed in recent work from Meta [9], [28], high coverage does not necessarily correlate with strong defect detection capability. Mutation testing offers a more rigorous alternative by assessing how well tests detect injected faults (mutants) [18]. Although some prior work [23], [28] has attempted to improve mutation scores during test generation, it does not fully explore the underlying limitations of LLMs in detecting and killing live mutants. Our work aims to address this gap by incorporating mutation feedback into the prompt design process for LLMs, thereby enhancing the effectiveness of the generated test cases. In addition, we introduce an iterative generation process to push the limits of LLMs in killing mutants, providing insights into their current capabilities and boundaries. This exploration also highlights opportunities for future work on guiding LLMs to generate test cases with stronger defect detection capabilities.

Other techniques. Automatic unit test generation has long been a central research topic in software engineering. Traditional efforts have primarily focused on search-based and random testing techniques, with tools such as EVOSUITE [29] and Randoop [53] significantly reducing the developers' manual testing burden. While EVOSUITE was originally designed to maximize code coverage, prior work [31] has investigated enhancing mutation scores by modifying its fitness functions, which is an objective closely related to our work. We conducted experiments along these lines and discuss the results in Section IV.B. In short, results are not significantly different from those of the default setting of EVOSUITE.

With the advent of large language models (LLMs), a new paradigm has emerged in test generation. Numerous recent surveys have summarized the rapid advancements in this area [27], [37], [66]; however, due to the large volume of related work, we only highlight a selection of representative examples in this section. Early research explored fine-tuning LLMs for code-related tasks. As LLMs became more powerful and general-purpose, however, prompt-based approaches gained traction, enabling test generation without task-specific training. Several studies [45], [57], [61], [71]–[73] have investigated prompt engineering strategies and mechanisms to improve test case generation effectiveness using commercial LLMs, notably OpenAI's GPT series [7], [16], [52].

Given the high cost of commercial LLMs, and the increasing availability of high-quality open-source alternatives such as the Llama and DeepSeek series [33], [46], recent research has shifted toward optimizing prompt-based effectiveness for open-source models [24], [38], [43], [51], [68], [69]. These models are becoming popular due to their accessibility and effectiveness in code-related tasks.

LLM-based test generation has been applied across a wide range of testing scenarios. Some studies focus on multilingual test generation across languages such as Python [47], [63], [68], [70], Java [20], [35], [68], [73], and others [21], [47], [67], [68], [74]. Other works target specific testing goals, such as detecting particular bugs [28], [47], API testing [17], [25],

[43], [44], or generating test oracles [15], [26], [42]. LLMs have also been integrated with traditional testing techniques, including mocking [49], [76], evolutionary algorithms [62], and program analysis [34], [55], [56], [60]. Evaluation in these works typically emphasizes code coverage metrics, such as branch and line coverage.

Unlike prior work, our approach focuses on automatically improving the mutation score with LLMs, based on mutation testing feedback, since such a score better reflects the fault-detection capability of test cases than code coverage [18], [28]. As discussed in Section II, high code coverage does not necessarily imply strong fault detection [18]. However, as demonstrated in our experiments (Section IV-B), improving mutation score often leads to increased code coverage as a by-product.

VI. DISCUSSION

We briefly discuss the limitations of our current work and outline directions for future research.

As described in Section IV and V, our experiments were conducted on independent, method-level subjects. This setup showed how well LLMs perform in killing mutants when not being confounded by broader program structures or dependencies. However, when transitioning to more complex datasets such as Defects4J, additional challenges arise, particularly in resolving dependencies to generate executable and meaningful test suites. We leave the investigation of such datasets for future work.

In addition, while we use Llama-3.3 as the base model in this study, MUTGEN is inherently model-agnostic and does not rely on any specific LLM. Though our results demonstrate the effectiveness of the approach, other models could potentially yield even better results. A more comprehensive evaluation across different LLMs is also left as future work.

Finally, although MUTGEN achieves high mutation scores on the selected subjects, there remain open challenges in pushing these scores even higher. For example, guiding LLMs to effectively kill mutants produced by certain mutation operators, such as `VoidMethodCalls`, remains difficult. In our evaluation on HumanEval-Java, this operator had only a 69.2% killing ratio. Developing strategies to better handle such challenging mutants is an important direction for future research.

VII. CONCLUSION

In this paper, we propose MUTGEN, a mutation-guided, LLM-based approach for automatically generating unit tests with high fault detection capability. The core innovation of MUTGEN lies in incorporating mutation feedback into the prompt to help LLMs generate tests that kill more mutants. In addition, two components—code summarization and fixing—further enhance the effectiveness of the approach.

We evaluated MUTGEN on 104 subjects from the widely-used HumanEval-Java dataset and 100 subjects from Leetcode-Java, which we constructed from LeetCode algorithmic problem solutions. Experimental results show that MUTGEN significantly outperforms both EVOSUITE and Llama-3.3 using a vanilla prompting strategy, achieving mutation scores of 89.5% and 89.1% on HumanEval-Java and Leetcode-Java, respectively. Since MUTGEN aims to maximize the mutation score of generated test cases, we also compare it with a variant of EvoSuite (EVOSUITE$_{\text{mut}}$), where the fitness function of maximizing coverage is replaced with one that maximizes mutation score. The results show that EVOSUITE$_{\text{mut}}$ performs similarly to EVOSUITE, but both are outperformed by MUTGEN.

Moreover, MUTGEN employs an iterative generation process to push the limits of LLMs in killing mutants. We further analyze the causes of remaining live and uncovered mutants, as well as the impact of different mutation operators, offering insights and directions for future research.

REFERENCES

[1] Description on mutation operators, Accessed: 2025.

[2] Jacoco, Accessed: 2025.

[3] Leetcode, Accessed: 2025.

[4] Mutahunter, Accessed: 2025.

[5] Ollama, Accessed: 2025.

[6] Pitest, Accessed: 2025.

[7] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[8] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. A3test: Assertion-augmented automated test case generation. *Information and Software Technology*, 176:107565, 2024.

[9] Nadia Alshahwan, Jubin Chheda, Anastasia Finegenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. Automated unit test improvement using large language models at meta. corr abs/2402.09171 (2024). *arXiv preprint arXiv:2402.09171*, 10, 2024.

[10] Saswat Anand, Edmund K Burke, Tsong Yueh Chen, John Clark, Myra B Cohen, Wolfgang Grieskamp, Mark Harman, Mary Jean Harrold, Phil McMinn, Antonia Bertolino, et al. An orchestrated survey of methodologies for automated software test case generation. *Journal of systems and software*, 86(8):1978–2001, 2013.

[11] James H Andrews, Lionel C Briand, Yvan Labiche, and Akbar Siami Namin. Using mutation analysis for assessing and comparing testing coverage criteria. *IEEE Transactions on Software Engineering*, 32(8):608–624, 2006.

[12] James H Andrews, Tim Menzies, and Felix CH Li. Genetic algorithms for randomized unit testing. *IEEE Transactions on Software Engineering*, 37(1):80–94, 2011.

[13] Andrea Arcuri and Lionel Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd international conference on software engineering*, pages 1–10, 2011.

[14] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868*, 2022.

[15] Soneya Binta Hossain and Matthew Dwyer. Togll: Correct and strong test oracle generation with llms. *arXiv e-prints*, pages arXiv–2405, 2024.

[16] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[17] Clinton Cao, Annibale Panichella, and Sicco Verwer. Automated test-case generation for rest apis using model inference search heuristic. *arXiv preprint arXiv:2412.03420*, 2024.

[18] Thierry Titcheu Chekam, Mike Papadakis, Yves Le Traon, and Mark Harman. An empirical study on mutation, statement and branch coverage fault revelation that avoids the unreliable clean program assumption. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, pages 597–608. IEEE, 2017.

[19] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[20] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, pages 572–576, 2024.

[21] Xiang Cheng, Fan Sang, Yizhuo Zhai, Xiaokuan Zhang, and Taesoo Kim. Rug: Turbo llm for rust unit test generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 634–634. IEEE Computer Society, 2025.

[22] Fulvio Corno, Ernesto Sánchez, Matteo Sonza Reorda, and Giovanni Squillero. Automatic test program generation: a case study. *IEEE Design & Test of Computers*, 21(2):102–109, 2004.

[23] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C Desmarais. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology*, 171:107468, 2024.

[24] Amirhossein Deljouyi, Roham Koohestani, Maliheh Izadi, and Andy Zaidman. Leveraging large language models for enhancing the understandability of generated unit tests. *arXiv preprint arXiv:2408.11710*, 2024.

[25] Sida Deng, Rubing Huang, Man Zhang, Chenhui Cui, Dave Towey, and Rongcun Wang. Lrasgen: Llm-based restful api specification generation. *arXiv preprint arXiv:2504.16833*, 2025.

[26] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*, pages 2130–2141, 2022.

[27] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53. IEEE, 2023.

[28] Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, and Shubho Sengupta. Mutation-guided llm-based test generation at meta. *arXiv preprint arXiv:2501.12862*, 2025.

[29] Gordon Fraser and Andrea Arcuri. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, pages 416–419, 2011.

[30] Gordon Fraser and Andrea Arcuri. A large-scale evaluation of automated unit test generation using evosuite. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 24(2):1–42, 2014.

[31] Gordon Fraser and Andrea Arcuri. Achieving scalable mutation-based generation of whole test suites. *Empirical Software Engineering*, 20(3):783–812, 2015.

[32] Shuzheng Gao, Chaozheng Wang, Cuiyun Gao, Xiaoqian Jiao, Chun Yong Chong, Shan Gao, and Michael Lyu. The prompt alchemist: Automated llm-tailored prompt optimization for test case generation. *arXiv preprint arXiv:2501.01329*, 2025.

[33] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[34] Sijia Gu, Noor Nashid, and Ali Mesbah. Llm test generation via iterative hybrid program analysis. *arXiv preprint arXiv:2503.13580*, 2025.

[35] Siqi Gu, Chunrong Fang, Quanjun Zhang, Fangyuan Tian, Jianyi Zhou, and Zhenyu Chen. Improving llm-based unit test generation via template-based repair. *arXiv preprint arXiv:2408.03095*, 2024.

[36] Richard Hamlet. Random testing. *Encyclopedia of software Engineering*, 2:971–978, 1994.

[37] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79, 2024.

[38] Dong Huang, Jie M Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv preprint arXiv:2312.13010*, 2023.

[39] Laura Inozemtseva and Reid Holmes. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th international conference on software engineering*, pages 435–445, 2014.

[40] Md Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. Mapcoder: Multi-agent code generation for competitive problem solving, 2024. *URL https://arxiv. org/abs/2405*, 11403.

[41] René Just, Darioush Jalali, and Michael D Ernst. Defects4j: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*, pages 437–440, 2014.

[42] Shaker Mahmud Khandaker, Fitsum Kifetew, Davide Prandi, and Angelo Susi. Augmentest: Enhancing tests with llm-driven oracles. *arXiv preprint arXiv:2501.17461*, 2025.

[43] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. Llamaresttest: Effective rest api testing with small language models. *arXiv preprint arXiv:2501.08598*, 2025.

[44] Jia Li, Jiacheng Shen, Yuxin Su, and Michael R Lyu. Llm-assisted mutation for whitebox api testing. *arXiv preprint arXiv:2504.05738*, 2025.

[45] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 14–26. IEEE, 2023.

[46] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[47] Kaibo Liu, Yiyang Liu, Zhenpeng Chen, Jie M Zhang, Yudong Han, Yun Ma, Ge Li, and Gang Huang. Llm-powered test case generation for detecting tricky bugs. *arXiv preprint arXiv:2404.10304*, 2024.

[48] Kshirasagar Naik and Priyadarshi Tripathy. *Software testing and quality assurance: theory and practice*. John Wiley & Sons, 2011.

[49] Zifan Nan, Zhaoqiang Guo, Kui Liu, and Xin Xia. Test intention guided llm-based unit test generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 779–779. IEEE Computer Society, 2025.

[50] Catherine Oriat. Jartege: a tool for random generation of unit tests for java classes. In *International Conference on the Quality of Software Architectures*, pages 242–256. Springer, 2005.

[51] Wendkûuni C Ouédraogo, Kader Kaboré, Haoye Tian, Yewei Song, Anil Koyuncu, Jacques Klein, David Lo, and Tegawendé F Bissyandé. Large-scale, independent and comprehensive study of the power of llms for test case generation. *arXiv preprint arXiv:2407.00225*, 2024.

[52] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[53] Carlos Pacheco and Michael D Ernst. Randoop: feedback-directed random testing for java. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, pages 815–816, 2007.

[54] Carlos Pacheco, Shuvendu K Lahiri, Michael D Ernst, and Thomas Ball. Feedback-directed random test generation. In *29th International Conference on Software Engineering (ICSE'07)*, pages 75–84. IEEE, 2007.

[55] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. Multi-language unit test generation using llms. *arXiv preprint arXiv:2409.03093*, 2024.

[56] Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. Aster: Natural and multi-language unit test generation with llms. *arXiv preprint arXiv:2409.03093*, 2025.

[57] Juan Altmayer Pizzorno and Emery D Berger. Coverup: Coverage-guided llm-based test generation. *arXiv preprint arXiv:2403.16218*, 2024.

[58] José Miguel Rojas, José Campos, Mattia Vivanti, Gordon Fraser, and Andrea Arcuri. Combining multiple coverage criteria in search-based unit test generation. In *Search-Based Software Engineering: 7th International Symposium, SSBSE 2015, Bergamo, Italy, September 5-7, 2015, Proceedings 7*, pages 93–108. Springer, 2015.

[59] José Miguel Rojas, Gordon Fraser, and Andrea Arcuri. Seeding strategies in search-based unit test generation. *Software Testing, Verification and Reliability*, 26(5):366–401, 2016.

[60] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. *Proceedings of the ACM on Software Engineering*, 1(FSE):951–971, 2024.

[61] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinícius

Carvalho Lopes. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, pages 313–322, 2024.

[62] Hamed Taherkhani, Melika Sepindband, Hung Viet Pham, Song Wang, and Hadi Hemmati. Epic: Cost-effective search-based prompt engineering of llms for code generation. *arXiv preprint arXiv:2408.11198*, 2024.

[63] Zhao Tian, Junjie Chen, and Xiangyu Zhang. Fixing large language models' specification misunderstanding for better code generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 645–645. IEEE Computer Society, 2025.

[64] András Vargha and Harold D Delaney. A critique and improvement of the cl common language effect size statistics of mcgraw and wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000.

[65] Mattia Vivanti, Andre Mis, Alessandra Gorla, and Gordon Fraser. Search-based data-flow test generation. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, pages 370–379. IEEE, 2013.

[66] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 2024.

[67] Wenhan Wang, Xuan Xie, Yuheng Huang, Renzhi Wang, An Ran Chen, and Lei Ma. Fine-grained testing for autonomous driving software: a study on autoware with llm-driven unit testing. *arXiv preprint arXiv:2501.09866*, 2025.

[68] Yibo Wang, Congying Xia, Wenting Zhao, Jiangshu Du, Chunyu Miao, Zhongfen Deng, Philip S Yu, and Chen Xing. Projecttest: A project-level unit test generation benchmark and impact of error fixing mechanisms. *arXiv preprint arXiv:2502.06556*, 2025.

[69] Zhijie Wang, Zijie Zhou, Yuheng Huang Da Song, Shengmai Chen, Lei Ma, and Tianyi Zhang. Towards understanding the characteristics of code generation errors made by large language models. *Preprint*, 2025.

[70] Jiacheng Xu, Bo Pang, Jin Qu, Hiroaki Hayashi, Caiming Xiong, and Yingbo Zhou. Clover: A test case generation benchmark with coverage, long-context, and verification. *arXiv preprint arXiv:2502.08806*, 2025.

[71] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, et al. On the evaluation of large language models in unit test generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 1607–1619, 2024.

[72] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering*, 1(FSE):1703–1726, 2024.

[73] Quanjun Zhang, Ye Shang, Chunrong Fang, Siqi Gu, Jianyi Zhou, and Zhenyu Chen. Testbench: Evaluating class-level test case generation capability of large language models. *arXiv preprint arXiv:2409.17561*, 2024.

[74] Yuwei Zhang, Qingyuan Lu, Kai Liu, Wensheng Dou, Jiaxin Zhu, Li Qian, Chunxi Zhang, Zheng Lin, and Jun Wei. Citywalk: Enhancing llm-based c++ unit test generation via project-dependency awareness and language-specific knowledge. *arXiv preprint arXiv:2501.16155*, 2025.

[75] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5673–5684, 2023.

[76] HENGCHENG ZHU, VALERIO TERRAGNI, LILI WEI, SHING-CHI CHEUNG, JIARONG WU, and YEPANG LIU. Understanding and characterizing mock assertions in unit tests. 2025.